



Systems and Network Administration



Chapter 2: Interoperability of Systems, Redundancy, and Load Balancing

Networks and Embedded Systems – 1st

2025-2026

Learning Objectives

- ✓ Understand the principles of system interoperability and how standards, protocols, and middleware enable communication between heterogeneous systems.
- ✓ Describe the role of redundancy in ensuring availability and fault tolerance at hardware, network, and application levels.
- ✓ Familiarize with key load balancing concepts



Introduction

- Modern computing infrastructures are no longer isolated entities; they are *ecosystems* of interconnected systems that must cooperate, recover from failure, and scale under unpredictable demand.
- Whether deployed in enterprise data centers, cloud environments, or hybrid architectures, these systems must exhibit three fundamental properties: **interoperability**, **redundancy**, and **load balancing**.
- Together, they ensure that services can communicate seamlessly, remain available despite faults, and handle variable workloads efficiently.

The Triad of Resilient System Design

- These three domains, *interoperability*, *redundancy*, and *load balancing*; form the foundation of resilient and scalable networked systems:
 - **Interoperability** enables heterogeneous components (hardware, operating systems, applications, and protocols) to exchange information and services. It is the antidote to technological fragmentation, ensuring that legacy systems and modern APIs coexist productively.
 - **Redundancy** provides continuity of service in the presence of hardware, software, or network failures. By duplicating critical components and introducing failover mechanisms, redundancy transforms single points of failure into recoverable events.
 - **Load balancing** distributes client requests and computational tasks among multiple servers or network links to maximize throughput, reduce latency, and prevent overload.
- These concepts are not independent; they reinforce each other. A truly robust infrastructure integrates all three:
 - ❑ Interoperability allows diverse systems to collaborate, redundancy guarantees availability, and load balancing maintains performance as demand fluctuates.

The Evolution of System Interconnection

- The journey from standalone computing to globally distributed architectures has been defined by an ever-growing need for systems to communicate.
- In the past, **monolithic systems** dominated enterprise computing, often using proprietary communication interfaces. As distributed computing matured in the 1990s, open standards such emerged to allow interaction between heterogeneous systems.
- These early frameworks laid the groundwork for today's service-oriented paradigms; **SOAP (Simple Object Access Protocol)** and **REST (Representational State Transfer)**, which define standardized, interoperable interfaces over the web.
- Today, **microservices**, **containers**, and **API gateways** dominate modern architectures. Each microservice, often written in a different language or hosted on a different platform, must communicate through well-defined APIs and message formats.
- This heterogeneity is both a *challenge* and a *strength*
- Interoperability allows organizations to integrate best-of-breed technologies while maintaining consistent operation across diverse environments.

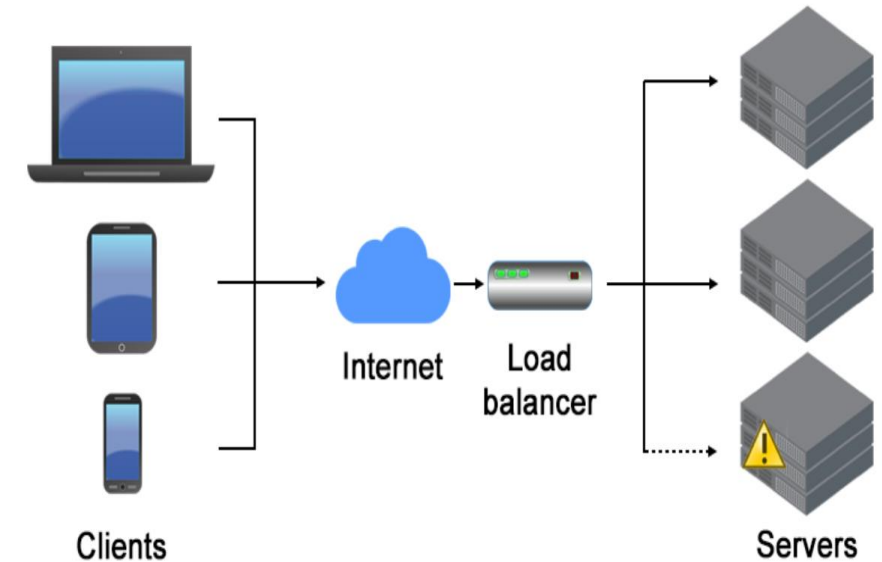
The Necessity of Redundancy

- As reliance on digital infrastructure has grown, so has the cost of downtime. Even a few minutes of service unavailability can translate into significant financial loss or reputational damage.
 - Redundancy addresses this risk by ensuring **no single component failure can bring the system down**.
 - Redundancy can appear at many levels:
 - Hardware redundancy,
 - Network redundancy
 - Application redundancy
 - Database redundancy
- ❑ The design principle is straightforward yet profound: *assume failure, then engineer recovery*.



The Role of Load Balancing in Modern Systems

- Where redundancy ensures continuity, **load balancing** ensures efficiency.
- A load balancer acts as the *traffic controller* of distributed systems, monitoring the health of servers and intelligently directing client requests to the most suitable instance.
- Proper load balancing leads to:
 - **Performance optimization**, by preventing resource exhaustion.
 - **Scalability**, by adding or removing servers without service disruption.
 - **Fault tolerance**, through automatic rerouting of traffic away from failed instances.



Chapter Overview

In the following sections, we will explore:

- ✓ The definition and implementation of interoperability, from data formats to middleware frameworks.
- ✓ Redundancy models across hardware, network, and application layers, including hands-on examples.
- ✓ Load balancing algorithms and configurations, from round robin to consistent hashing.

Part 1: System Interoperability

Definition and Core Principles

- **System interoperability** is the capability of distinct computing systems, software applications, and network services to **exchange data and use that information meaningfully**.
- The term encompasses both *syntactic* and *semantic* compatibility:
 - **Syntactic interoperability** ensures that systems can *communicate structurally*, that data formats, protocols, and message structures are understood on both sides.
 - **Semantic interoperability** guarantees that systems *interpret exchanged data consistently*, that a “user ID,” for example, means the same thing to every component.

Definition and Core Principles

- True interoperability thus requires a shared understanding of:
 - **Data representation** (e.g., XML, JSON, YAML)
 - **Communication protocols** (e.g., HTTP, gRPC, MQTT)
 - **Service contracts and APIs**
 - **Security and identity models** (e.g., OAuth 2.0, OpenID Connect)
 - **Governance and compatibility rules** (versioning, backward compatibility)
- Modern distributed systems achieve this through **open standards, middleware, and API-driven communication**.
- The guiding principle, as defined in ISO/IEC 2382 and IEEE Std 610, is that interoperable systems *can exchange information and make use of it cooperatively, in a way that requires little to no user knowledge of their unique characteristics*.

Layers of Interoperability

- Interoperability can be analyzed in **four layers**:

Layer	Description	Example Technologies
<i>Data layer</i>	Defines how information is represented and exchanged.	JSON, XML, YAML, CSV
<i>Protocol layer</i>	Specifies how systems communicate.	HTTP, TCP/IP, gRPC, MQTT
<i>Service layer</i>	Describes callable operations and interfaces.	REST APIs, SOAP services, GraphQL
<i>Semantic layer</i>	Defines shared vocabularies and meaning.	RDF, OWL, domain ontologies, JSON Schema

- Each layer contributes to the overall goal: enabling different systems (possibly built decades apart) to work together as if they were part of a single coherent network.

Communication Models and Standards

Remote Procedure Call (RPC)

- **RPC** allows a program to execute procedures on a remote system as if they were local. The client sends a request specifying the function and parameters; the server executes it and returns the result. Key protocols:
 - **ONC RPC (RFC 5531)** : early UNIX implementation.
 - **DCE/RPC** : basis for Microsoft's DCOM.
 - **gRPC (Google RPC)** : a modern high-performance framework using HTTP/2 and Protocol Buffers.

- **gRPC Example (Python):**

```
# greet.proto
syntax = "proto3";
service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}
message HelloRequest { string name = 1; }
message HelloReply { string message = 1; }
```

- ❑ This definition is compiled into client and server code in multiple languages, ensuring **interoperability by design**. gRPC also supports streaming, deadlines, and authentication via TLS or OAuth 2.0.

SOAP (Simple Object Access Protocol)

- SOAP, standardized by W3C, uses **XML-based messages** and typically relies on HTTP or SMTP transport. It defines strict message envelopes, headers, and encoding rules.
- SOAP emphasizes **formal contracts** defined by WSDL (Web Services Description Language).
- Example fragment:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">  
  <soap:Body>  
    <GetWeather xmlns="http://example.com/weather">  
      <City>London</City>  
    </GetWeather>  
  </soap:Body>  
</soap:Envelope>
```

- SOAP was central to **enterprise interoperability** during the early 2000s but is often considered heavyweight compared to newer approaches like REST.

REST (Representational State Transfer)

- **REST** is an architectural style formalized by Roy Fielding (2000) in his doctoral dissertation. It emphasizes:
 - **Stateless communication**
 - **Uniform interfaces** (HTTP verbs: GET, POST, PUT, DELETE)
 - **Resource identification via URIs**
 - **Use of standard media types (JSON, XML)**
- Example REST interaction:

Request

```
GET /api/users/42 HTTP/1.1
Accept: application/json
```

Response

```
{
  "id": 42,
  "username": "user",
  "email": « user@example.com"
}
```

GraphQL and Beyond

- GraphQL, introduced by Facebook in 2015, allows clients to specify exactly what data they need.
- Unlike REST's fixed endpoints, GraphQL uses a **single flexible query endpoint**.

```
{  
  user(id: 42) {  
    username  
    posts(limit: 2) {  
      title  
    }  
  }  
}
```

- This approach improves efficiency and reduces over-fetching, making it popular for complex, data-rich applications.

Middleware and Integration Frameworks

- Middleware acts as the **glue** that connects incompatible systems.
- It abstracts communication details, manages message transformation, and provides services like authentication and monitoring.
- Common middleware categories:
 - **Message-oriented middleware (MOM)**: RabbitMQ, Kafka, ActiveMQ.
 - **Enterprise Service Bus (ESB)**: Mule ESB, Apache ServiceMix.
 - **API gateways and service meshes**: Kong, Istio, Envoy.
- For instance, **Apache Kafka** supports interoperability by decoupling producers and consumers through a publish-subscribe model. Data is exchanged asynchronously in a distributed, fault-tolerant cluster, an ideal foundation for microservice communication.

Interoperability in Cloud and Hybrid Environments

- In modern infrastructure, interoperability extends beyond local systems to **multi-cloud** and **hybrid** deployments.
- Tools like **Terraform**, **Ansible**, and **Kubernetes** use declarative specifications (YAML, JSON) to ensure that workloads can move between environments without modification.
- Cloud providers now expose **standardized APIs** (OpenStack, AWS EC2, Azure ARM, Google Cloud API) and support **interoperable container formats** (OCI-compliant Docker images).
- Interoperability here is essential for **portability**, **vendor neutrality**, and **resilience**.

Security and Governance

- Interoperable systems must not compromise security.
- Standards like **TLS 1.3 (RFC 8446)**, **OAuth 2.0**, and **OpenID Connect** ensure secure communication and federated identity management across platforms.
- Governance frameworks, such as **API versioning policies**, **access control lists (ACLs)**, and **zero-trust architectures**, maintain consistency and compliance.

Part 2: Redundancy

Definition and Objectives

- **Redundancy** in systems and network administration refers to the *intentional duplication of critical components or functions* of a system to increase reliability and availability.
- Its goal is not waste but **resilience**: the capacity to continue operating correctly despite failures.
- A redundant architecture ensures that:
 - No single failure can bring the system down (**no single point of failure**, or SPOF).
 - Service continuity is preserved through **failover mechanisms**.
 - Maintenance or upgrades can occur **without service interruption**.
 - The system can **detect, isolate, and recover** from faults automatically.
- Redundancy embodies the principle “*design for failure*” , assuming that every hardware, software, and network element will eventually fail

Types of Redundancy

Level	Description	Example Technologies
<i>Hardware Redundancy</i>	Duplicate physical components.	RAID arrays, dual PSUs, ECC memory
<i>Network Redundancy</i>	Duplicate paths or interfaces.	Link aggregation, VRRP, HSRP
<i>Software / Application Redundancy</i>	Redundant processes or containers.	Cluster managers, Kubernetes replicas
<i>Data Redundancy</i>	Multiple copies of information.	Database replication, distributed filesystems
<i>Geographical Redundancy</i>	Independent sites in different regions.	Multi-region cloud clusters, CDN replication

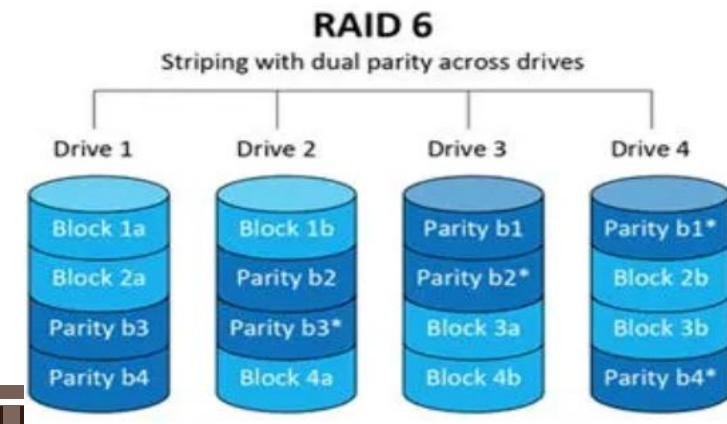
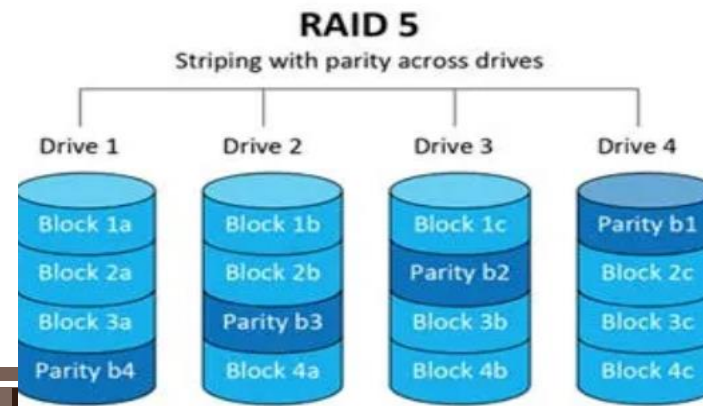
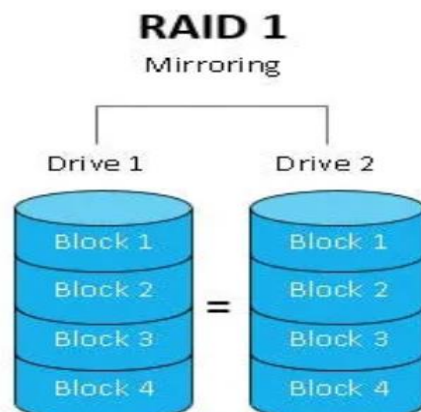
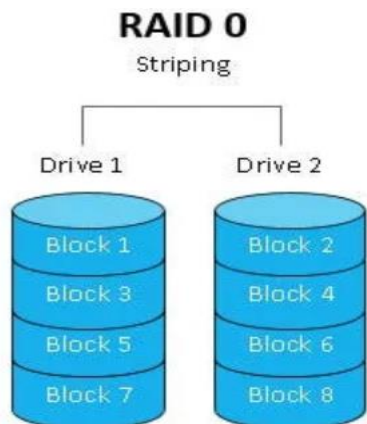
Each layer builds upon the one below it to achieve full **High Availability (HA)**.

Hardware Redundancy

Redundant Arrays of Independent Disks (RAID)

- RAID combines multiple drives into logical arrays that provide **fault tolerance and performance improvements**. Examples include :

RAID Level	Mechanism	Fault Tolerance	Typical Use
<i>RAID 0</i>	Striping (no redundancy)	0 drives	High-performance scratch data
<i>RAID 1</i>	Mirroring	1 drive failure	Critical system disks
<i>RAID 5</i>	Striping + parity	1 drive failure	General servers
<i>RAID 6</i>	Double parity	2 drive failures	Storage arrays



Hardware Redundancy

- Example (Linux *mdadm* configuration):

Create a RAID1 array from two disks

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sda1 /dev/sdb1
```

```
sudo mkfs.ext4 /dev/md0
```

- **Hot-spare drives** further enhance redundancy by automatically replacing failed disks. A hot spare is a drive that's not used until a disk fails. When one disk in the array dies, the hot spare automatically takes its place, and RAID rebuilds itself using parity or mirroring, with no manual intervention.

#This adds /dev/sdc1 as a hot spare to the existing RAID array /dev/md0.

```
sudo mdadm --add /dev/md0 /dev/sdc1
```

Power and Cooling Redundancy

- Servers in production data centers use **dual power supplies (PSUs)** connected to **separate power feeds** or UPS units.
- Cooling fans and HVAC systems are similarly duplicated in an $N + 1$ configuration, meaning one additional unit beyond the minimum needed to operate.



Network Interface Card (NIC) Bonding

- NIC teaming or bonding combines multiple network interfaces into one logical link for both redundancy and throughput.
- Example:

```
auto bond0
iface bond0 inet static
    address 10.0.0.10
    netmask 255.255.255.0
    bond-mode active-backup
    bond-slaves eth0 eth1
    bond-miimon 100
```

- If ***eth0*** fails, traffic automatically shifts to ***eth1***.

Network Redundancy

Routing Protocols and High Availability

- Routers and gateways must maintain connectivity even when hardware fails.
- **VRRP (Virtual Router Redundancy Protocol)**, defined in RFC 5798, creates a *virtual IP address* shared by a group of routers. If the master fails, a backup automatically takes over within seconds.

✓ Example (*Keepalived* configuration on Linux):



```
vrrp_instance VI_1 {  
    state MASTER  
    interface eth0  
    virtual_router_id 51  
    priority 100  
    advert_int 1  
    virtual_ipaddress {  
        192.168.1.1  
    }  
}
```

❑ In a backup node, the priority is set lower (e.g., 90).

Network Path Redundancy

- **Spanning Tree Protocol** (*STP, IEEE 802.1D*): prevents loops in redundant Ethernet topologies.
- **Equal-Cost Multipath Routing** (*ECMP*): distributes traffic across multiple paths of equal metric.
- **MPLS Fast Reroute**: provides link-failure recovery in under 50 milliseconds in carrier networks.

Software and Application Redundancy

Clustering

- **Application clustering** groups multiple servers to act as a single logical service. Clustering can be:
 1. **Active-Passive:** only one node handles requests; others wait for failover.
 2. **Active-Active:** all nodes serve requests simultaneously, sharing load and state.
- Linux HA tools like **Pacemaker + Corosync** implement heartbeat monitoring and automatic failover.
- ✓ Example (cluster resource definition):

```
pcs resource create webserver ocf:heartbeat:apache op monitor interval=30s
```

Container and Orchestrator Redundancy

- In containerized environments, redundancy is handled at the orchestration layer.
- **Kubernetes** ensures that a desired number of pod replicas are always running.
- If a node crashes, pods are rescheduled elsewhere.

✓ *Example of Kubernetes Deployment Manifest*



- ❑ This configuration guarantees three instances of the application are always available.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx
          ports:
            - containerPort: 80
```

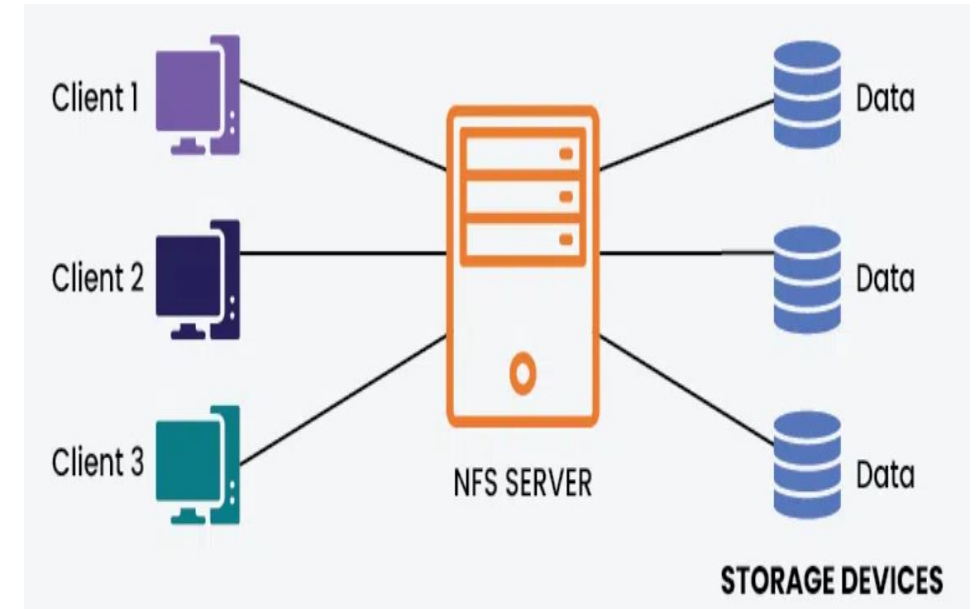
Data and Database Redundancy

Replication Models

- Database redundancy maintains data availability and consistency across nodes.
- Common replication models:
 - **Master-Slave Replication:** writes occur on one primary node; replicas serve reads.
 - **Master-Master Replication:** all nodes accept writes; requires conflict resolution.
 - **Synchronous Replication:** transactions commit only after acknowledgment from replicas.
 - **Asynchronous Replication:** replicas lag slightly behind the primary but offer better performance.
 - **Quorum-based Replication:** used in distributed databases (Cassandra, Etcd, Raft) to achieve consensus.

Distributed File Systems

- Distributed file systems are designed to **store data across multiple servers (nodes)** and automatically handle:
 - **Replication** : multiple copies of data blocks.
 - **Fault tolerance** : if one node or disk fails, data is still accessible from another replica.
 - **Scalability** : capacity and performance grow as more nodes are added.



Distributed File Systems

- Examples of Distributed File Systems :

System	Description	Typical Use
<i>Ceph</i>	Object, block, and file storage system; uses OSDs (Object Storage Daemons) for distributed data replication.	Cloud storage, OpenStack, Kubernetes storage backends.
<i>GlusterFS</i>	Scales out by combining storage across multiple servers into one large networked filesystem.	General-purpose file storage and media repositories.
<i>HDFS (Hadoop Distributed File System)</i>	Core of Hadoop; splits big data into blocks and replicates them across cluster nodes.	Big data analytics and processing.

Testing and Monitoring Redundancy

- Redundancy is useful only if it works when needed.
- Therefore:
 - **Simulated failures** (*chaos engineering*): validate resilience.
 - **Health checks**: detect unavailable nodes for automatic failover.
 - **Monitoring systems**: like Prometheus, Zabbix, and Nagios collect metrics on uptime and replication lag.
 - **Logging and alerting**: integrations (Slack, PagerDuty, Email) enable rapid response.

Part 3:

Load Balancing

Introduction

- As systems scale, redundancy alone is not enough. Having multiple servers is valuable only if requests can be distributed among them intelligently.
- **Load balancing** is the process of distributing client requests or network traffic across multiple servers to ensure optimal resource utilization, minimal response time, and fault tolerance.
- A **load balancer (LB)** acts as the *traffic coordinator* of distributed systems; sitting between clients and servers, determining which server should handle each request based on various criteria such as load, proximity, or health status.
- Load balancing exists at both:
 - **Layer 4 (Transport layer):** based on IP address and TCP/UDP ports.
 - **Layer 7 (Application layer) :** based on content (HTTP headers, cookies, URLs, etc.).
- By combining redundancy and load balancing, systems achieve both **availability** and **scalability**.

Goals and Benefits of Load Balancing

- ✓ **High Availability:** If one server fails, the load balancer redirects traffic to healthy instances.
- ✓ **Scalability:** New servers can be added seamlessly without downtime.
- ✓ **Performance Optimization:** Requests are spread evenly to prevent overload.
- ✓ **Maintainability:** Individual servers can be taken offline for updates or repairs without affecting service.
- ✓ **Security and Abstraction:** The LB hides backend infrastructure, filtering malicious requests or throttling abusive clients.

Types of Load Balancers

Type	Layer	Example Tools	Description
Hardware LB	L4/L7	F5 BIG-IP, Citrix NetScaler	Dedicated appliances for large-scale enterprises.
Software LB	L4/L7	HAProxy, NGINX, LVS, Envoy	Open-source and flexible; used in most modern setups.
DNS-based LB	L3	BIND, Route 53, Cloudflare DNS	Distributes requests via multiple A/AAAA records.
Anycast LB	L3	BGP Anycast	Routes clients to the nearest server via IP routing.
Application-aware LB	L7	Envoy, Traefik, Istio	Integrated into microservice meshes.

Load Balancing Algorithms

- The algorithm determines how incoming requests are distributed among backend servers.

Algorithm	Description	Use Case
<i>Round Robin</i>	Each request is sent to the next server in order.	Simple, uniform workloads.
<i>Weighted Round Robin</i>	Servers get requests based on assigned weights.	Mixed-capacity servers.
<i>Least Connections</i>	New requests go to the server with the fewest active connections.	Variable-length sessions (e.g., web apps).
<i>IP Hash (Consistent Hashing)</i>	Clients are mapped to servers using a hash of their IP.	Session persistence (sticky sessions).
<i>Random</i>	Requests distributed randomly.	High volume with low variance.
<i>Least Response Time</i>	Chooses the server with lowest latency and fewest connections.	Latency-sensitive systems.
<i>Custom / Dynamic</i>	Decisions based on custom metrics (CPU, memory, latency).	Adaptive systems, cloud autoscaling.

Layer 4 vs. Layer 7 Load Balancing

Feature	Layer 4 (Transport)	Layer 7 (Application)
Routing basis	IP, TCP/UDP ports	URLs, cookies, headers
Performance	Faster (less inspection)	Slightly slower (content parsing)
Flexibility	Limited	High, can rewrite requests
Examples	Linux IPVS, LVS, MetalLB	HAProxy, NGINX, Envoy
Use cases	Database, DNS, streaming	Web apps, APIs, microservices

- In practice, many infrastructures use a **two-tier approach**:
 - L4 load balancer (e.g., AWS ELB, IPVS, or cloud LB) distributes connections to nodes.
 - L7 proxy (e.g., HAProxy or NGINX) routes traffic intelligently inside each node.

Health Checks and Failover

- A key feature of load balancers is **continuous health monitoring**. If a backend fails to respond, it is temporarily removed from the rotation.

✓ Example HAProxy configuration:



```
backend web_servers
  balance roundrobin
  option httpchk GET /health
  server web1 10.0.0.1:80 check
  server web2 10.0.0.2:80 check
```

- ❑ If `/health` on web2 returns a 500 error or times out, traffic is automatically redirected to web1 until recovery.

Session Persistence (“Sticky Sessions”)

- Some applications, especially **stateful web applications**, need a client to consistently connect to the **same backend server** across multiple requests.
- This is crucial when:
 - Session data (like shopping carts, login tokens, or in-progress forms) is **stored in memory** on one server,
 - And not yet shared via a centralized session store (like *Redis* or a database).
- Without persistence, requests from the same user might land on different backends, causing them to “lose” session data.

Common Methods for Session Persistence

Method	Mechanism	Notes
IP Hashing	The load balancer uses the client's IP address to choose the backend.	Simple but not ideal when clients share IPs (e.g., behind NAT).
Cookie-based Stickiness	The load balancer injects or tracks a special cookie that identifies the chosen backend.	Most common for HTTP; works well even behind NAT or proxies.
Session Affinity via Tokens	The application itself issues a session token (e.g., JWT) that encodes which backend to use.	Used in advanced setups with app-level control.

SSL/TLS Termination

- In secure environments, load balancers often handle **SSL/TLS encryption** to offload CPU overhead from backend servers. This is known as **SSL termination**.

- Example (NGINX):



```
server {  
    listen 443 ssl;  
    ssl_certificate  
    /etc/nginx/ssl/cert.pem;  
    ssl_certificate_key  
    /etc/nginx/ssl/key.pem;  
    location / {  
        proxy_pass  
        http://backend_cluster;  
    }  
}
```

- This allows backends to communicate over plain HTTP, simplifying management and improving performance.

Scaling Strategies

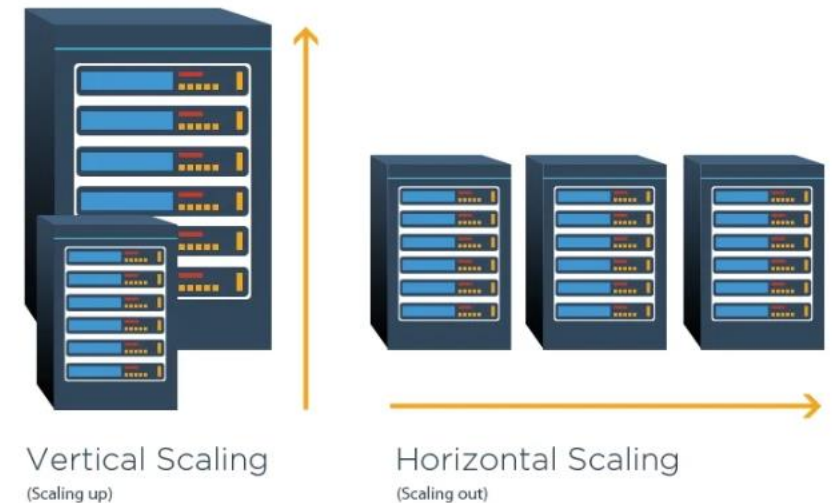
- **Vertical vs. Horizontal Scaling:**

- **Vertical scaling:** increasing resources of a single server (CPU, RAM).
- **Horizontal scaling:** adding more servers to distribute load.

❑ Load balancing enables **horizontal scalability**, scaling out dynamically under pressure and scaling in when load decreases.

- **Auto Scaling:**

- Cloud providers (AWS, Azure, GCP) integrate load balancers with auto-scaling groups.
- Metrics like CPU usage or request rate trigger automatic instance creation or deletion.



Software Load Balancers

- **HAProxy (High Availability Proxy)** is one of the most widely used open-source L7 load balancers.
- It supports HTTP/TCP balancing, health checks, SSL termination, and advanced routing.

✓ Minimal configuration example:



```
frontend http_front
  bind *:80
  default_backend web_servers

backend web_servers
  balance leastconn
  server web1 10.0.0.1:80 check
  server web2 10.0.0.2:80 check
```

Software Load Balancers

- **NGINX :**

- Originally a web server, **NGINX** is also a capable load balancer and reverse proxy.

- Config Example :



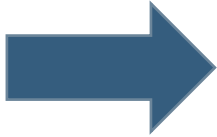
```
upstream backend {  
    least_conn;  
    server 10.0.0.10;  
    server 10.0.0.11;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://backend;  
    }  
}
```

Software Load Balancers

▪ Linux Virtual Server (LVS):

- **LVS** operates at Layer 4 using IPVS (IP Virtual Server) kernel module.
It offers high performance and is often used for massive-scale systems.

✓ Example (*ipvsadm* command):



```
ipvsadm -A -t 192.168.1.100:80 -s rr  
ipvsadm -a -t 192.168.1.100:80 -r 192.168.1.1:80 -g  
ipvsadm -a -t 192.168.1.100:80 -r 192.168.1.2:80 -g
```

- *ipvsadm -A -t 192.168.1.100:80 -s rr*
 - Adds a virtual service listening on IP 192.168.1.100, port 80, using **round-robin** scheduling (-s rr).
- *ipvsadm -a -t 192.168.1.100:80 -r 192.168.1.1:80 -g*
 - Adds a real backend server 192.168.1.1 in **gateway (direct routing)** mode (-g).
- *ipvsadm -a -t 192.168.1.100:80 -r 192.168.1.2:80 -g*
 - Adds another backend server in the same mode.

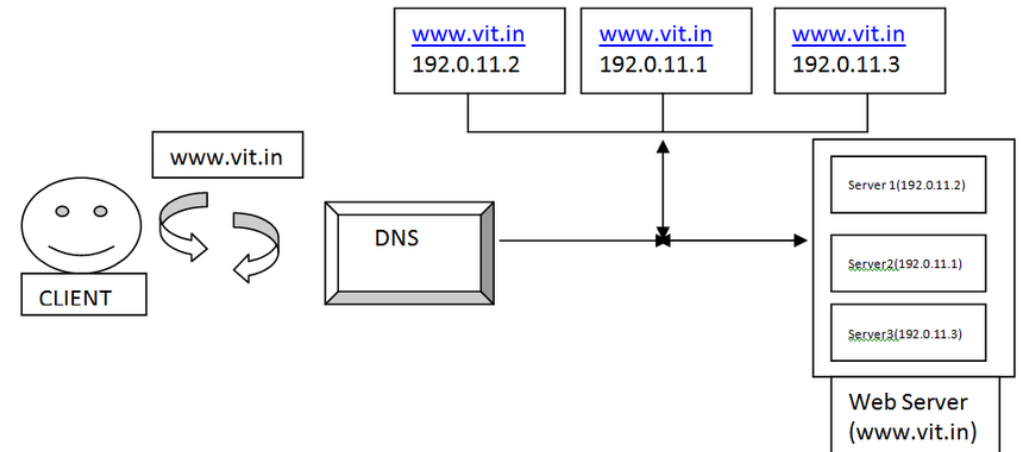
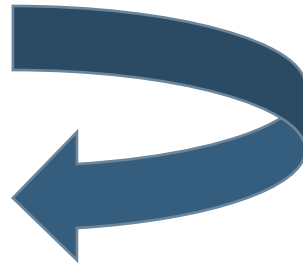
Distributed and Global Load Balancing

DNS-based Load Balancing

- Multiple IP addresses are assigned to the same domain.
- Clients are distributed via round-robin DNS or geographic routing (GeoDNS).
 - Limitations: DNS caching and slow failover.

✓ Example (BIND zone file):

```
www IN A 192.0.2.1  
www IN A 192.0.2.2
```



Distributed and Global Load Balancing

Anycast Load Balancing:

- **Anycast** is a **network-level (Layer 3)** load balancing and routing technique.
- The **same IP address** is announced (advertised) from **multiple geographically distributed servers** or data centers.
- Routing is handled by **BGP (Border Gateway Protocol)**, not by a traditional load balancer.
- When a client sends traffic to that IP, the Internet's routing system automatically directs it to the **“nearest” node**, where *nearest* means the lowest BGP routing cost (shortest AS path), **not necessarily physical distance**.

